

Installing Spark: Download the standalone cluster version of Spark from its website <http://spark.apache.org/downloads.html>

Then extract the file by typing the following command in the terminal:

```
$ tar xvf spark-2.0.0-bin-hadoop2.6.tgz
```

Add entry to `.bashrc` by:

```
nano ~/.bashrc
```

Add the line specifying the location to `~/.bashrc` by:

```
export SPARK_HOME=/home/sapna/spark-2.0.0-bin-hadoop2.6/
export PATH=$PATH:$SPARK_HOME/bin
```

Then source it by using the command below:

```
$ source ~/.bashrc
```

Start Spark services and the shell. Then let's change the directory by going into Spark's folder and manually starting the master cluster using the command shown below:

```
cd spark-2.0.0-bin-hadoop2.6
./sbin/start-master.sh
```

After running this, you can view the user interface of the master node by typing the following command in the browser:

```
http://localhost:8080
```

You can start the slave node by giving the following command:

```
./sbin/start-slave.sh <name of slave node to run>
```

To check if the nodes are running, execute the following:

```
Jps
```

Architecture of the Apache Spark engine

Spark uses a master/worker architecture. There is a driver called the *Spark Context object*, which interacts with a single coordinator called the *master* that manages workers in which executors run.

Spark is founded on two chief concepts—the RDD (Resilient Distributed Dataset) and DAG (Directed Acyclic Graph) execution engine. An RDD, a read-only immutable collection of objects, is the basic data structure of Spark. The data is partitioned, and each RDD can be computed on a different node and can be written in many languages. It stores the state of the memory as an object across the jobs

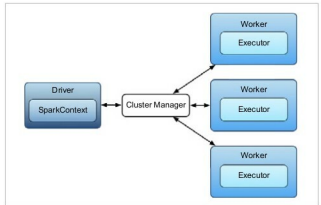


Figure 1: Architecture of the Spark engine

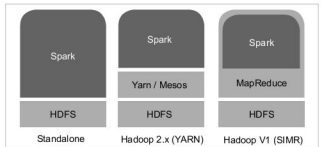


Figure 2: Possible deployment scenarios for the Spark engine

and the object is shareable between those jobs. RDD can transform data by mapping or filtering it, or it can perform operations and return values. RDDs can be parallelised and are intrinsically fault-tolerant. They can be created through two methods—by taking an existing collection in your driver application and parallelising it or by creating a reference from an external storage system like HDFS, HBase, AWS, etc. The DAG helps to obviate the multi-staged model of MapReduce which offers processing advantages.

Spark can be deployed in three popular ways to cater to different scenarios. The first way is to use a standalone mode. Here, Spark is placed above HDFS and allocates memory to it manually. All Spark jobs on the clusters are executed with Spark and MapReduce running simultaneously. The second way is to use a cluster management system such as Hadoop YARN (Yet Another Resource Manager), which doesn't require any pre-installation or root access to integrate with the Hadoop stack or ecosystem. Other components can be externally added on top to increase the functionality. The third way is to use SIMR (Spark in MapReduce) which, in addition to a manager, also executes a Spark job. Spark shell can be used without any administrative authorisation.

The main elements that constitute Spark are: Spark Core, MLlib, GraphX, Spark Streaming and Spark SQL. Spark Core is the basic platform engine that serves as a foundation for building other functionalities. The Spark SQL component, which provides the abstraction called SchemaRDD, which allows the loading, analysis and processing of semi-structured